

Course Title: Machine Learning Sessional

Course Code: CSE 442

Experiment: Classification

Project: News Classification Using Machine Learning (TF-IDF with Logistic Regression, Naive Bayes and Linear SVM)

Submitted To:

Sumaiya Rahman Aodhora

Lecturer

Department of Computer Science and Engineering

Submitted By:

Sadia Akter

ID: 2022-1-17-011

Department of Computer Science and Engineering

Submission Deadline: 17 / 11 / 2026

1. Project Title

News Classification Using Machine Learning (TF-IDF with Logistic Regression, Naive Bayes and Linear SVM)

2. Objective

The main goal of this project is to build a computer program that can read a short news article and tell which category it belongs to. The four categories are **Business, SciTech (Science & Technology), Sports and World**. The small goals of the project are:

- To clean the news text before using it.
- To convert the text into numbers using TF-IDF.
- To train three machine learning models: Logistic Regression, Naive Bayes, and Linear SVM.
- To compare the accuracy of the three models and find the best one.
- To test the best model with some new news sentences.

3. Apparatus

- **Computer or Laptop**
- **Google Colab:** To write and run the Python code and build Development Environment
- **Google Drive:** To store the dataset file and connect it with Colab
- **Python 3:** Main Programming Language
- **Python Libraries:**
  - **Pandas:** For Reading And Handling The Dataset
  - **Numpy**: For Numerical Calculations And Array Operations
  - **Matplotlib:** To draw graphs and charts
  - **NLTK:** To get the list of English stop words
  - **Scikit-learn:** To build the models, split data and measure results
- **Input Dataset:** NewsClassificationDataSet.csv

4. Theory

4.1 Text Classification: Text classification means placing a text into a group or category. In this project, each news article is placed in one of four categories. This is supervised learning because I train the model using articles that already have correct categories.

4.2 Text Preprocessing: News text may contain extra marks and common words that are not useful. I clean the text before training the model -

- **Lowercase:** I change all letters to small letters. For example, “Apple” becomes “apple”.
- **Remove punctuation:** I remove marks such as commas, full stops, and question marks.
- **Remove stop words:** I remove common words such as “the”, “is”, and “of” because they usually do not show the news topic.

4.3 Label Encoding: Computers work with numbers, so I change the category names into numbers:

- Business = 0,
- SciTech = 1,
- Sports = 2, and

- World = 3.

4.4 Train–Test Split: I divide the dataset into two parts. I use 80% of the articles to train the model and 20% to test it. The stratify option keeps the same share of each category in both parts.

4.5 TF-IDF (Term Frequency – Inverse Document Frequency): TF-IDF changes words into numbers. It gives more importance to words that are useful for one article but do not appear in every article. For example, "goal" may be useful for Sports news. I used single words and two-word groups, and selected up to 10,000 features.

4.6 Machine Learning Models

1. **Logistic Regression:** It calculates the possibility of each category and selects the category with the highest score.
2. **Multinomial Naive Bayes:** It uses probability to predict a category from the words in an article. It is simple and fast, and is often useful for text classification.
3. **Linear SVM (Linear Support Vector Machine):** It finds the best boundary to separate the different news categories.

4.7 Evaluation Measures

- **Accuracy:** The percentage of all predictions that are correct.
- **Precision:** Out of the articles predicted as a category, how many truly belong to that category.
- **Recall:** Out of all articles that really belong to a category, how many the model correctly found.
- **F1-score:** A score that combines precision and recall.
- **Confusion Matrix:** A table that shows correct predictions and the categories the model confused.

5. Dataset Description

My dataset file is named NewsClassificationDataSet.csv. Each line contains one news article and its category. The article text is stored under "content", and the category is stored under "annotation → label". I use Python's json library to read this file line by line.

Item	Details
Total articles	7,600
Columns used	text (news article), label (category)
Categories	Business, SciTech, Sports, World
Articles per category	1,900 each (perfectly balanced)
Missing values	0 (none)
Training data	6,080 articles (80%)
Testing data	1,520 articles (20%)

Each category has the same number of articles. So, the dataset is balanced, and accuracy is useful for comparing the models.

6. Methodology

I completed this project in Google Colab using Python. The main steps of the code are explained below.

Step 1: Import Libraries: I bring in all the tools we need. Pandas handles data, Matplotlib draws charts, NLTK gives stop words, and Scikit-learn gives the models and the measuring tools. Warnings are hidden to keep the output clean.

Step 2: Load the Dataset: Google Drive is connected to Colab. The file is read one line at a time. From each line I take the news text and its first label, and put them in a table (DataFrame) with two columns: `text` and `label`.

Step 3: Check the Data and Remove Missing Values: I look at the data type, count missing values, and count the articles in each category. Any row with a missing text or label is removed. In my data nothing was missing, so all 7,600 rows stayed.

Step 4: Draw the Class Distribution Chart: A bar chart shows how many articles each category has. It helps us see that the data is balanced.

Step 5: Clean the Text: The function clean_text makes the text lowercase, removes punctuation, splits it into words, and throws away stop words. Then it joins the words again. I apply it to every article and save the result in a new column called clean_text.

Step 6: Encode the Labels: LabelEncoder changes the category names to numbers (0, 1, 2, 3) so the models can use them.

Step 7: Split into Training and Testing Data: 80% of the data is used for training and 20% for testing. random_state=42 gives the same split every time I run the code. stratify=y keeps the same category share in both parts.

Step 8: TF-IDF Vectorization: This step changes the text into numbers. max_features=10000 keeps the top 10,000 words/word-pairs. ngram_range=(1, 2) uses single words and two-word pairs. min_df=2 ignores terms that appear in fewer than 2 articles. max_df=0.95 ignores terms that appear in more than 95% of articles. The vectorizer learns only from the training data (fit_transform) and then only converts the test data (transform), so there is no cheating.

Step 9: Train the Three Models: For each model I do three things: fit (learn from the training data), predict (guess the category of the test articles), and accuracy_score (compare the guesses with the real labels).

Step 10: Compare the Models and Find the Best One: The accuracy of all three models is put in one table and shown in a bar chart. The code then automatically picks the model with the highest accuracy as the best model.

Step 11: Classification Reports and Confusion Matrices: The classification report gives precision, recall and F1-score for each category. The confusion matrix shows how many articles were correctly and wrongly predicted in each category. I did this for all three models.

Step 12: Predict the Category of New News: The function predict_news takes any new sentence, cleans it the same way as the training data, converts it with the same TF-IDF, and predicts the category with the best model. Finally, inverse_transform changes the number back into a category name. I tested it with four new sentences.

Step 13: Final Summary: This prints a short final result of the whole project.

7. Visualization

7.1 News Category Distribution

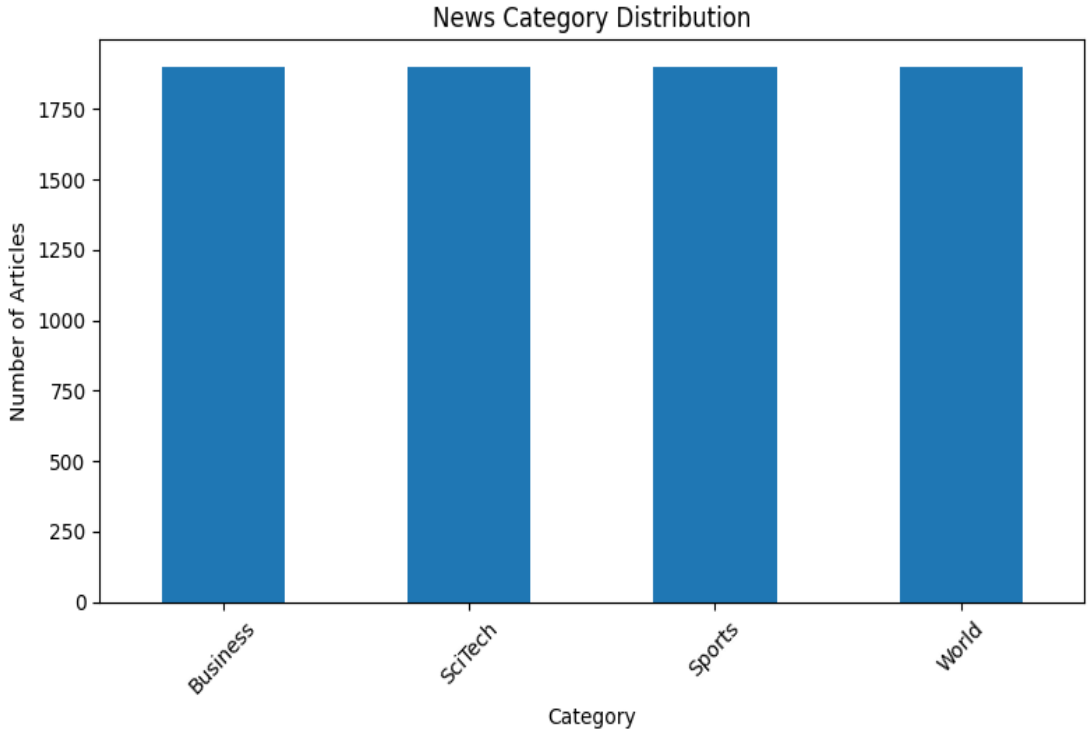


Figure 1: News Category Distribution. Every category has 1,900 articles.

7.2 Accuracy Comparison

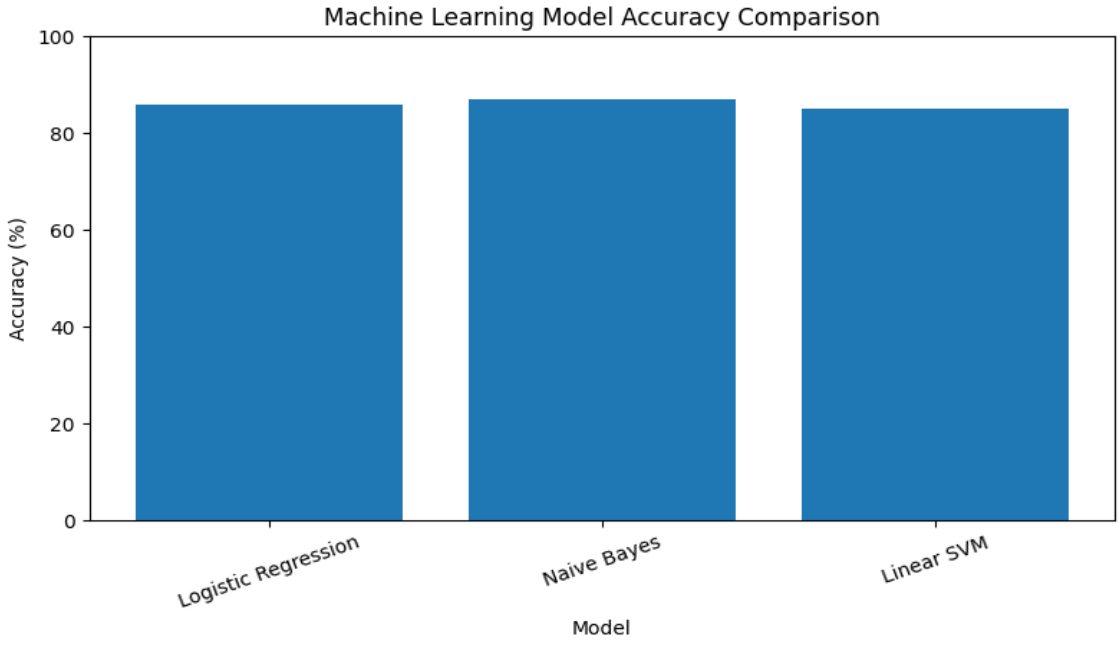


Figure 2: Accuracy comparison of the three models.

7.3 Confusion matrices

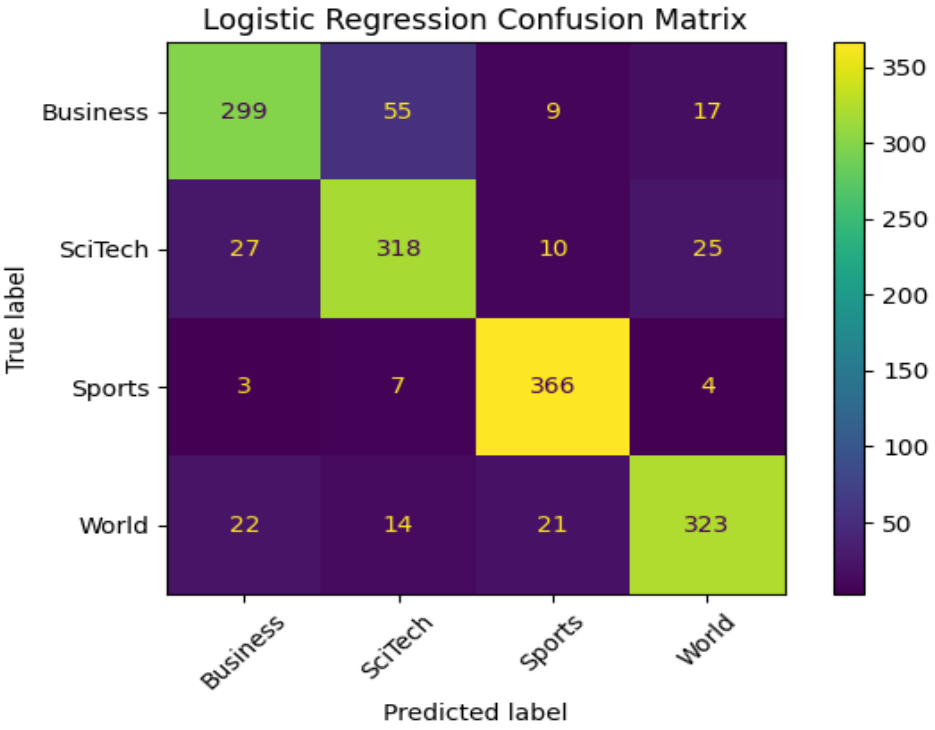


Figure 3: Logistic Regression confusion matrix.

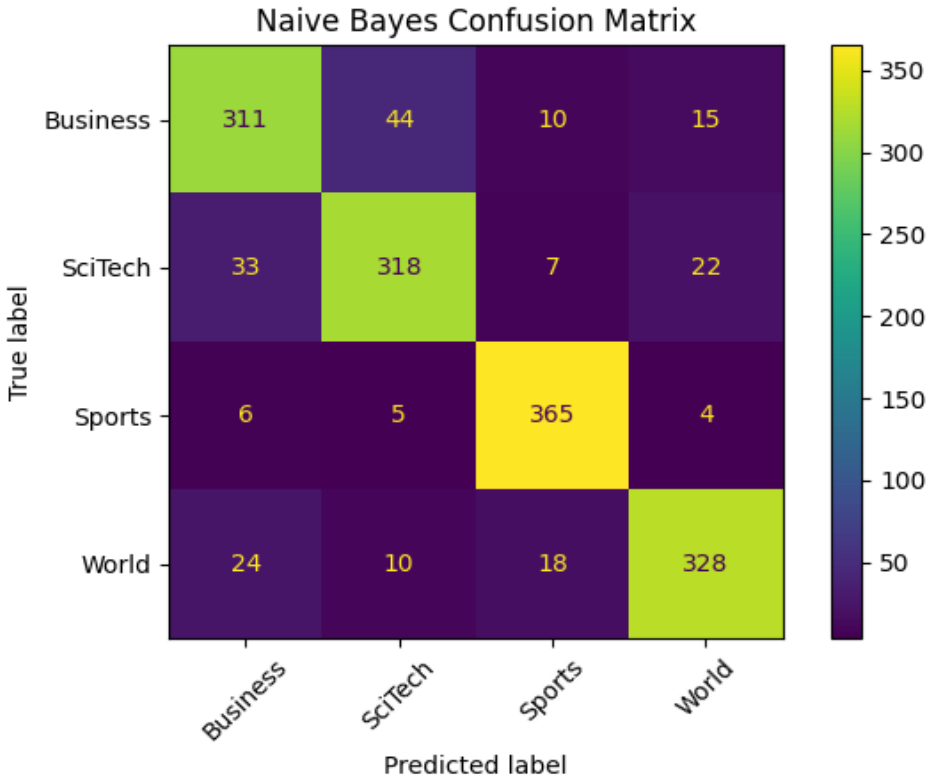


Figure 4: Naive Bayes confusion matrix.

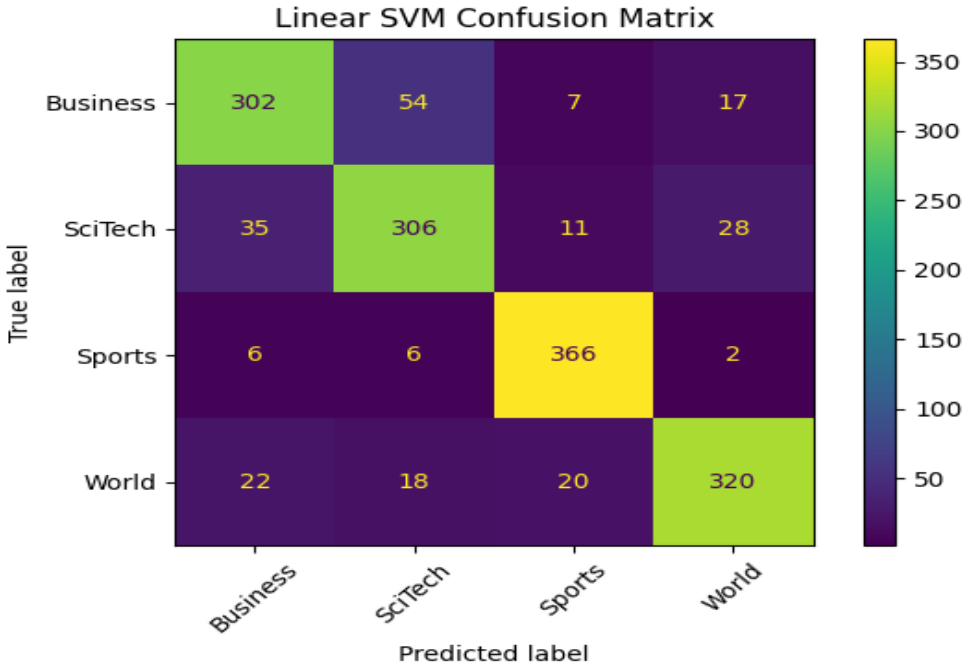


Figure 5: Linear SVM confusion matrix.

8. Result

8.1 Model Accuracy

1. Logistic Regression Accuracy: 85.92%
2. Naive Bayes Accuracy: 86.97%
3. Linear SVM Accuracy: 85.13%

8.2 Model Comparison

	Model	Accuracy	Accuracy (%)
0	Logistic Regression	0.859211	85.921053
1	Naive Bayes	0.869737	86.973684
2	Linear SVM	0.851316	85.131579

8.3 Classification Reports (Precision / Recall / F1-Score)

Category	LR Precision	LR Recall	LR F1	NB Precision	NB Recall	NB F1	SVM Precision	SVM Recall	SVM F1
Business	0.85	0.79	0.82	0.83	0.82	0.82	0.83	0.79	0.81
SciTech	0.81	0.84	0.82	0.84	0.84	0.84	0.80	0.81	0.80
Sports	0.90	0.96	0.93	0.91	0.96	0.94	0.91	0.96	0.93
World	0.88	0.85	0.86	0.89	0.86	0.88	0.87	0.84	0.86
Overall Accuracy			**0.86**			**0.87**			**0.85**

Here, LR = Logistic Regression, NB = Naive Bayes, SVM = Linear SVM and Each category has 380 test articles.

8.4 Final Summary

NEWS CLASSIFICATION PROJECT RESULTS

Logistic Regression: 85.92%
Naive Bayes: 86.97%
Linear SVM: 85.13%

Best Model: Naive Bayes
Best Accuracy: 86.97%

9. Discussion

- **Best Model:** Naive Bayes performed best with 86.97% accuracy. The other two models also gave good results.
- **Why Naive Bayes:** It works well with text and words, so it performed better in this project.
- **Category Results:** Sports was the easiest category to predict. Business and SciTech were more difficult.
- **New News Test:** The model correctly predicted all four new sentences.
- **Good Points:** The dataset was balanced, and all models were tested using the same data.
- **Limitations:** I tested only one data split and four new sentences. I did not try different model settings.
- **Future Work:** I can use Cross-Validation, GridSearchCV, LSTM, and BERT to improve the results. I can also build a website for news prediction.

10. Conclusion

In this project, I developed a machine learning system to classify news into four categories: Business, SciTech, Sports, and World. I used 7,600 news articles and cleaned the text. Then I converted the text into numbers using TF-IDF. I tested three models: Logistic Regression, Naive Bayes, and Linear SVM. Naive Bayes gave the best result with 86.97% accuracy. The system could identify Sports news more easily than Business and SciTech news. I also tested four new sentences, and the best model predicted all of them correctly. Overall, this project shows how machine learning can help us classify news automatically. In the future, I want to improve the system using more data and advanced models.